TCP Echo 服务器系列开发

序言

具体项目代码可以看我的TCP-echo-Server仓库

本文记录了我在 Linux 环境下，使用 C 语言分多个阶段实现一个 TCP Echo 服务器。从最基础的单客户端 TCP 服务器，逐步迭代到基于 Epoll 的高并发服务器的全过程，分析了各版本的实现逻辑、优缺点与性能差异，旨在系统性梳理网络 I/O 编程的核心知识。每个阶段需独立编译运行，并通过多客户端测试验证。

前置学习：gcc、Makefile、git

🟢 第一阶段：单客户端同步版本 (The Basics)

目标：掌握 Socket 基本 API 和 TCP 通信流程

功能要求：

1. 创建 Socket，绑定端口（如 8080），开始监听。
2. 调用 `accept()` 等待一个客户端连接。
3. 进入循环：调用 `recv()` 接收数据，立即调用 `send()` 原样返回。
4. 当客户端断开连接（`recv` 返回 0）时，关闭当前连接并退出程序（或重新回到 `accept` 等待下一个，但不支持同时连接）。

考察点：

- `socket`，`bind`，`listen`，`accept`，`recv`，`send`，`close` 的正确使用。
- 基本的错误处理（检查返回值）。
- 自测方法：启动服务器，用 `telnet 127.0.0.1 8080` 连接，发送文字看是否回显。开启第二个终端尝试连接，应发现第二个连接被阻塞（排队），直到第一个断开。

实现逻辑：

- 流程：`socket() → bind() → listen() → accept() → recv() → send() → close()`
- 阻塞模式：
  - `accept()` 阻塞等待客户端连接
  - `recv()` 阻塞等待数据
- 同一时间**只能处理一个客户端**

🟡 第二阶段：多线程同步版本 (Concurrency with Threads)

目标：理解并发处理，学习 POSIX 线程 (pthread)。

功能要求：

1. 在主线程中循环调用 `accept()` 接收新连接。
2. 关键变化：每当 `accept()` 成功返回一个新连接描述符后，创建一个新线程 (`pthread_create`) 专门负责该连接的 `recv` / `send` 循环。
3. 主线程继续回去 `accept` 下一个连接，实现“来者不拒”。
4. 子线程在客户端断开后，自行清理资源并退出。

考察点：

- `pthread_create` , `pthread_detach` (或 `pthread_join`) 的使用。
- 线程函数参数的传递（如何把 `client_socket` 安全传给线程）。
- 自测方法：启动服务器，开启 5-10 个终端窗口同时 `telnet` 连接。所有窗口应能同时收发数据，互不阻塞。

实现逻辑：

主线程只负责 `accept` ，每接入一个新连接，就 `pthread_create` 一个新线程专门负责该连接。线程独立处理 recv/send，并采用线程分离（pthread_detach）自动释放资源

但是线程是昂贵的资源（每个线程约占 8MB 栈空间），当连接数达到几千个时，内存会爆掉，CPU 会在频繁的线程切换中精疲力竭。

🔴 第三阶段：Select 多路复用版本 (I/O Multiplexing)

目标：理解 I/O 多路复用机制，在一个线程内管理多个连接。

功能要求：

1. 不再使用多线程，整个程序只有一个主线程。
2. 维护一个文件描述符集合 (`fd_set`)。
3. 使用 `select()` 系统调用监控：监听套接字（是否有新连接）和 所有已连接的客户端套接字（是否有数据可读）。
4. 当 `select` 返回就绪时：
   - 如果是监听套接字就绪 -> 调用 `accept` 加入集合。
   - 如果是客户端套接字就绪 -> 调用 `recv` 读取并 `send` 返回。若客户端断开，将其从集合中移除并关闭。

考察点：

- `fd_set`，`FD_ZERO`，`FD_SET`，`FD_ISSET` 宏的使用。
- `select` 函数的参数构造及超时设置。
- 理解为什么 `select` 能在一行代码中监控多个文件描述符。
- 局限性思考：让实习生思考 `select` 的文件描述符数量限制（通常是 1024）。
- 自测方法：同样开启多个客户端连接，验证并发正常。观察系统资源占用（应比多线程版本更轻量，没有大量线程上下文切换）。

实现逻辑：引入 `fd_set` 位图，通过 `select` 函数将一堆 Socket 交给内核监控。

但位图长度固定，效率低下复杂度为O(n)，每次都要全量遍历看哪个fd动了，且需要手动更新 `max_fd`。

⚫ 第四阶段：Epoll版本

`epoll` 不像 `select` 和 `poll` 那样需要轮询所有文件描述符，而是通过事件通知机制，只有当文件描述符上有事件发生时才会通知进程。且不再需要维护 `max_fd`，直接操作就绪数组。

epoll主要有两种工作模式：

- **水平触发（LT）**：当文件描述符上有数据可读/可写时，epoll_wait() 会持续返回该文件描述符，直到数据被完全读取/写入。只要条件满足（缓冲区有数据未读，或者可以写入数据），就会一直通知。优点是保证了数据不会丢失，实现简单，容错性好。但如果应用程序没有及时处理就绪事件，epoll_wait() 可能会频繁地返回同一个文件描述符，造成忙等待，降低效率。
- **边缘触发（ET）**：只有当文件描述符的状态发生变化时（从不可读变为可读，或者从不可写变为可写），epoll_wait() 才会返回该文件描述符。只有在状态改变的时候才会通知一次。优点是效率更高，因为只有状态变化时才会通知。但需要应用程序一次性读取/写入所有的数据，否则可能会丢失事件。 通常需要配合非阻塞 I/O 一起使用。

实现逻辑：使用 `epoll_create` 创建红黑树，`epoll_wait` 直接获取就绪列表

🔵 第五阶段：做一个Client（先做一个简单的半双工收发，再改为epoll）

懒得写了😋